Problem Set 1

Name: Nicolas Iskandar

Problem 1-1.

(a)

$$\begin{aligned} f_1 &= log(n^n) = n.log(n) && \implies O(nlog(n)) \\ f_2 &= (log(n))^n && \implies O(log^n(n)) \\ f_3 &= (log(n^6 006)) = 6006 \times log(n) && \implies O(log(n)) \\ f_4 &= (log(n))^6 006 && \implies O(log^{6006}(n)) \\ f_5 &= (log(log(6006n))) && \implies O(log(log(n))) \end{aligned}$$

$$\implies (f_5, f_3, f_4, f_1, f_2)$$

(b) $(f_1, f_2, f_5, f_4, f_3)$

(c) Approximate using Stirling

$$\begin{aligned} f_1 &\implies O(n^n) \\ f_2 &\implies O(n^6) \\ f_3 &\implies O(n^{6n}) \\ f_4 &\implies O(n^{\frac{2n}{3}}) \\ f_5 &\implies O(n^6) \end{aligned}$$

$$\implies (\{f_2, f_5\}, f_4, f_1, f_3)$$

(d) Approximate using Stirling

$$\begin{aligned} f_1 &\implies O(n^n) \\ f_2 &\implies O(n^{\sqrt{n)}} \\ f_3 &\implies O(4^{nlog(n)}) \\ f_4 &\implies O(7^{n^2}) \\ f_5 &\implies O(n^{\frac{1}{n}}) \end{aligned}$$

$$\implies (f_5, f_2, f_1, f_3, f_4)$$

Problem 1-2.

(a)

```
def reverse(D, i, k):
  left = i
  right = i + k - 1
  while left < right:
    x = D.delete_at(right)
    y = D.delete_at(left)

    D.insert_at(left, x)
    D.insert_at(right, y)
    left += 1
    right -= 1
```

(b)

```
def move(D, i, k, j):
  if j < i:
    for offset in range(k):
      x = D.delete_at(i + offset)
      D.insert_at(j + offset, x)

  else:  # j > i + k - 1
    for offset in range(k-1, -1, -1):
      x = D.delete_at(i + offset)
      D.insert_at(j - k + 1 + offset, x)
```

Problem 1-3.

- **build(X)**: Store all pages in a static array and initialize sections P1, P2, P3 with gaps in O(n).
- **place mark(i, m)**: Rebuild array S to insert bookmark m at index i, maintaining gaps between P1, P2, P3 in O(n).
- **read page(i)**: Compute which section (P1, P2, P3) contains index i and return page in O(1).
- **shift mark(m, d)**: Move bookmark by one position and adjust section indices while keeping gaps in O(1).
- **move page(m)**: Move the page at bookmark m to the other bookmark and rebuild if gaps close, O(1) amortized

Problem 1-4.

(a)
- **insert first(x):** Insert x at the beginning by setting $x.next$ to the current head, updating the current head's previous pointer to x if it exists, and then updating $L.head$ to x; if the list was empty, also set $L.tail$ to x.
- **insert last(x):** Insert x at the end by setting $x.prev$ to the current tail, updating the current tail's next pointer to x if it exists, and then updating $L.tail$ to x; if the list was empty, also set $L.head$ to x.
- **delete first():** Remove the first node by updating $L.head$ to $L.head.next$ and setting the new head's previous pointer to None if it exists; if the list becomes empty, set $L.tail$ to None.
- **delete last():** Remove the last node by updating $L.tail$ to $L.tail.prev$ and setting the new tail's next pointer to None if it exists; if the list becomes empty, set $L.head$ to None.

(b) **Remove sublist from x_1 to x_2:** Disconnect the sublist by setting $x_1.prev.next$ to $x_2.next$ (if x_1 is not the head) and $x_2.next.prev$ to $x_1.prev$ (if x_2 is not the tail). Update $L.head$ to $x_2.next$ **if** x_1 was the head, and $L.tail$ to $x_1.prev$ **if** x_2 was the tail. Set $x_1.prev$ to None and $x_2.next$ to None to isolate the sublist, then return it as a new doubly linked list with head x_1 and tail x_2.

(c) **Splice L_2 into L_1 after node x:** If L_2 is not empty, set $L_2.head.prev = x$ and $L_2.tail.next = x.next$; if $x.next$ exists, set $x.next.prev = L_2.tail$; then set $x.next = L_2.head$. If x was the tail of L_1, update $L_1.tail = L_2.tail$. Finally, set $L_2.head =$ None and $L_2.tail =$ None to empty L_2.